A PROJECT REPORT ON

Retro Game Hub

BACHELOR OF TECHNOLOGY IN CSIT (CYBER SECURITY)

Submitted By | Group No: 3

PRN	Name of the Student
2501106112	Dristhi Galani
2501106113	Tanvee Thakre
2501106118	Atharva Tirkhunde
2501106136	Rishit Kinge
2501106155	Shubham Salunke

Under the Guidance of Prof. Rahul Raut

CONTENTS

1 Introduction

2 Literature Review

3 Subject Integration Overview

4 Design and Methodology

5 System Architecture

6 Implementation

7 Results

8 Industry Relevance

9 Challenges

10 Ethical Considerations

11 Future Scope

12 Conclusion

13 References

INTRODUCTION

Background

Retro Game Hub is a browser-based arcade platform recreating classic notebook games — Tic-Tac-Toe, Dots & Boxes, Snakes & Ladders and more. Built using only HTML, CSS, and Vanilla JavaScript across 13 files (~4,200 lines) with zero external dependencies.

Motivation

Build a compact, offline-capable gaming platform showcasing game logic, UI design, LocalStorage state persistence, and AI via Minimax & Alpha-Beta pruning. Accessible on any device, anywhere — even without internet.

Problem Statement

Most retro games are isolated mobile apps — bloated with ads, requiring accounts and data connections. There is a gap for a lightweight, browser-accessible hub that runs offline on any device with a unified player profile system.

Scope

9 playable games · Player profiles with PIN auth · Coin reward economy · Per-game & global leaderboards · 4 retro display themes · Procedural sound engine (7 music tracks, 12 SFX) · Responsive UI (360 px – 1440 px)

OBJECTIVES

- Build a multi-game web app using only HTML, CSS & Vanilla JavaScript
- Implement persistent player profiles & reward tracking using LocalStorage
- Create AI opponents (Minimax, Alpha-Beta, Hunt/Target, Heuristic) for multiple games
- Provide a polished retro interface with sound, themes & responsive design

LITERATURE REVIEW

Single-Game Implementations

Focus on one classic title (e.g. standalone Tetris or Tic-Tac-Toe clones) with no unified player system or progression. Simple to host but no cohesion.

Framework-Dependent Portals

Multi-game portals built on React / Angular. Heavier footprint, harder to host as a single static folder, and dependency-bound.

Server-Dependent Platforms

Require a backend to store player data. Cannot work offline and need a persistent internet connection to function.

COMPARATIVE STUDY — Retro Game Hub vs Existing Solutions

Feature	Standalone Clones	Framework Portals	Server Platforms	Retro Game Hub
No install required	✓	✓	✓	✓
Multiple games	✗	✓	✓	✓
Works offline	Usually	✗	✗	✓
Player profiles	✗	Varies	✓	✓
AI opponents	Sometimes	Varies	Varies	✓
Zero dependencies	✓	✗	✗	✓

SUBJECT INTEGRATION OVERVIEW

CS23201 | Data Structures & Algorithms

Concept: Arrays, Stacks, Queues, Game Trees

Game state management, board representation (Tic-Tac-Toe, Connect Four), Minimax AI using tree structures

CS23202 | Web Technologies & Vulnerabilities

Concept: HTML, CSS, JS, LocalStorage, Web Security

Complete frontend UI, responsive design, secure PIN-based login, avoiding plain-text storage

CS23203 | OS, Compiler & Security Engineering

Concept: Process Flow, Memory Management, Security

Modular code structure, efficient game loops, hashed PIN storage

CS23204 | Human Behavior

Concept: UX, Cognitive Design, Interaction Principles

Intuitive UI, retro themes, sound feedback, engaging gameplay experience

CS23205 | Python Programming

Concept: Logic Building, Algorithm Design

Programming fundamentals (implemented in JS) — loops, conditionals, modular coding

CS23206 | Applied Mathematics - II

Concept: Probability, Logic, Graph Theory, Optimization

AI decision-making (Minimax), probability-based mechanics (dice), scoring/reward calculations

DESIGN AND METHODOLOGY

Development Approach

Iterative feature-branch workflow. Each game module built & tested in isolation before integration.

Sprint 1: SPA shell, player system, UI manager
Sprint 2: Tic-Tac-Toe, Dots & Boxes, Snakes & Ladders, coin system
Sprint 3: Battleship, Connect Four, Hangman, Memory Match
Sprint 4: Tetris, Pong + final bug-fix sprint

Tools & Technologies

HTML5 · CSS3 (custom properties) · ES6+ Vanilla JavaScript
Web Audio API (procedural sound) · LocalStorage API · SVG

Dev Tools: VS Code · Chrome DevTools · Git

SOFTWARE REQUIREMENTS

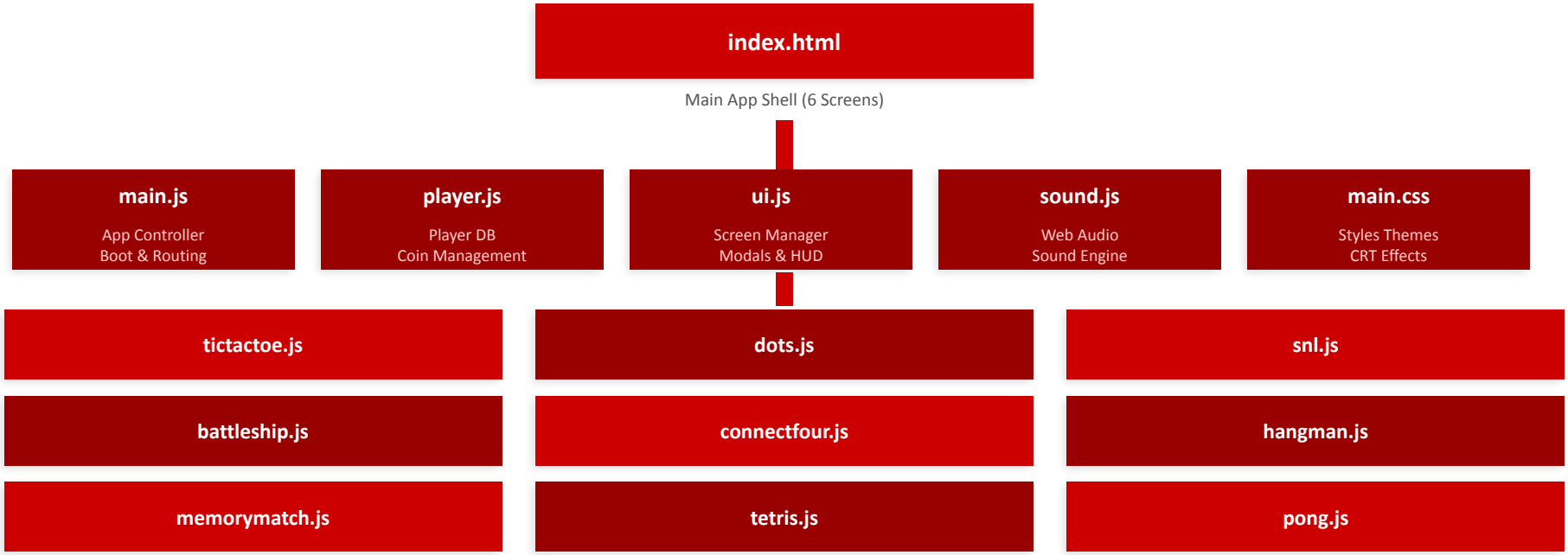
OS	Any OS with a modern browser (Windows, macOS, Linux, Android, iOS)
Browser	Chrome 100+ · Firefox 100+ · Edge 100+ · Safari 15+ · Samsung Internet 20+
JavaScript	Must be enabled (ES6+)
Server/DB	None — uses browser LocalStorage
Installation	None — open index.html directly
Internet	Not required after first load

SCORING & REWARD MODEL

- Win reward: 10–25 coins (fixed per game, varies by difficulty)
- Participation reward: +2 coins for any completed game (win or lose)
- Streak bonus: +5 coins for 3 consecutive wins

SYSTEM ARCHITECTURE

Modular Single-Page Application (SPA) Architecture



9 Individual Game Modules — each with own logic, AI, and reward handling

IMPLEMENTATION

KEY ALGORITHMS

Minimax (Tic-Tac-Toe)

Recursively explores every game state. Assigns +10 AI win / -10 AI loss / 0 draw. Always evaluates full tree (max 362,880 states) ⟶ unbeatable AI.

Alpha-Beta Pruning (Connect Four)

Minimax on 6×7 board, depth limited to 5. Pruning reduces nodes from O(b^d) to O(b^(d/2)). Heuristic scores centre control and open 2/3/4-disc sequences.

Hunt & Target (Battleship)

Hunt mode: fires at random non-adjacent cells. On hit ⟶ Target mode: fires at 4 orthogonal neighbours until ship sunk ⟶ reverts to Hunt.

Heuristic AI (Dots & Boxes)

3-tier strategy: (1) capture any completable box ⟶ (2) take safe line that won't give opponent a 3-sided box ⟶ (3) random fallback.

LocalStorage Player Schema (key: retro_players, indexed by player ID)

- id — Unique player ID (3–16 alphanumeric chars)
- pinHash — SHA-like hash of 4-digit PIN (never plain text)
- avatar — Chosen emoji (set of 24)
- coins — Running total
- wins/games — Per-game & total counts
- streak — Consecutive win count

RESULTS

13
Total Files

~4,200
Lines of Code

0
Dependencies

<200ms
Page Load Time

<30MB
Memory Usage

5
Supported Browsers

KEY TEST CASES & OUTCOMES

Test Case	Expected Result	Outcome
Player login and guest mode	Both modes accessible	✓ Pass
Coin award on Tetris win	+20 coins added	✓ Pass (after fix)
Coin award on Pong win	+12 coins added	✓ Pass (after fix)
Game-over modal for all games	Modal appears with result	✓ Pass
LocalStorage persistence	Coins survive page reload	✓ Pass
Offline use after first load	All features work	✓ Pass

INDUSTRY RELEVANCE

💻 Front-End Engineering

Demonstrates production-grade proficiency in HTML5, CSS3, and ES6+ JavaScript without any framework dependency — directly applicable to web development roles.

🤖 AI & Algorithm Design

Implementation of Minimax, Alpha-Beta pruning, and Hunt & Target AI reflects core CS fundamentals valued in software engineering and game development industries.

🔐 Cyber Security Principles

PIN hashing, avoiding plain-text storage, and modular security design align with CSIT programme outcomes and industry best practices for client-side security.

📱 PWA & Mobile Readiness

PWA manifest file and responsive layout (360 px–1440 px) position the app for Play Store distribution via Trusted Web Activity (TWA).

🎮 Game Development Patterns

State machines, game loops, canvas physics (Pong), and procedural audio via Web Audio API are patterns used directly in the indie game development industry.

☁️ Cloud & Scalability Path

Architecture is designed to scale — Phase 3 plans WebSocket multiplayer, Firebase/Supabase cloud sync, and tournament brackets, mirroring SaaS platform design.

CHALLENGES

Fragmented Coin Reward System

Problem: Each game handled outcomes independently, causing inconsistencies and bugs in coin allocation across the 9 games. **Solution:** Centralized reward via a unified PlayerDB.awardCoins() function followed by synchronized UI updates using UI.updateHUD().

Inconsistent Game-Over Handling

Problem: Different games triggered end-game states in different ways, leading to unpredictable modal behavior and missed coin credits. **Solution:** All game-ending scenarios standardized through a single modal system using UI.showWinModal(), improving reliability.

Duplicate Keyboard Listeners (Tetris)

Problem: On game restart, keyboard event listeners were registered a second time without removing the previous ones, causing double moves. **Solution:** Implemented proper listener de-registration before each game init to ensure a single active listener at all times.

Zero-Dependency Audio

Problem: Providing background music and SFX without any audio files or external libraries, while maintaining a rich multi-layered sound experience. **Solution:** Procedural audio synthesized entirely via Web Audio API oscillators — melody, bass, and harmony layers with 7 tracks and 12 SFX. Zero bytes of audio payload.

ETHICAL CONSIDERATIONS

Data Privacy & Security

Player PINs are never stored in plain text. A SHA-like hash is applied before saving to LocalStorage. No personal data is transmitted to any server — all data stays on the user's own device.

No Ads or Monetization Dark Patterns

The platform contains zero advertisements. The coin economy is transparent and non-deceptive. Guest mode ensures access to all games without forced registration or data collection.

Accessibility & Inclusivity

Responsive design supports screens from 360 px (budget phones) to 1440 px desktops. Works across 5 major browsers. Fully offline-capable, reducing the digital divide for users with limited internet.

Academic Integrity

All code is originally written by the team. External references (MDN, GeeksforGeeks, W3C specifications) are properly cited. The project declares no plagiarism and acknowledges all borrowed concepts.

Open-Source Community Respect

No external libraries were used, avoiding license compliance risks. Documentation and open learning resources that guided the project are acknowledged in the References section.

Responsible AI Design

AI opponents are deliberately balanced — not designed to frustrate users. Difficulty is appropriate to each game. AI is transparent in behaviour, with no deceptive or manipulative mechanics.

FUTURE SCOPE

PHASE 2 — Multiplayer & Expansion

- Real-time multiplayer via WebSocket communication with simple room codes
- New games: Rock-Paper-Scissors, Sudoku — expanding platform variety
- Progressive Web App (PWA) framework for mobile device installation

PHASE 3 — Gamification & Cloud

- Achievement badge system & daily challenge modes rotating every 24 hours
- Customizable in-game elements — unlockable skins purchasable with coins
- Global tournament brackets with up to 4-player multiplayer support
- Cloud-based data sync using Firebase / Supabase for cross-device profiles

DEPLOYMENT

- PWA manifest & icon assets already created — service worker implementation pending
- Google Play Store distribution planned via Trusted Web Activity (TWA)

CONCLUSION

Retro Game Hub successfully delivers a complete browser-based arcade — proving that core web technologies alone can power complex, interactive applications.

✓ 9 Fully Functional Games

All games playable with consistent design, interaction patterns, and AI opponents.

✓ Persistent Player Profiles

Coin balances and progress reliably persist across sessions via LocalStorage.

✓ Balanced AI Opponents

Minimax, Alpha-Beta, Hunt/Target & Heuristic AI provide engaging single-player challenge.

✓ Responsive Retro UI

Polished retro theme adapts seamlessly from 360 px mobile to 1440 px desktop.

✓ Zero Dependencies

Entire platform runs on HTML5, CSS3, and Vanilla JS — no framework, no server, no installs.

✓ Stable Architecture

Reward system, game-over modals, and keyboard listeners unified and bug-free after final sprint.

REFERENCES

[1] MDN Web Docs — HTML, CSS, and JavaScript Reference. Mozilla Developer Network. https://developer.mozilla.org

[2] Web Audio API Specification. W3C Working Draft. https://www.w3.org/TR/webaudio/

[3] Using the Web Storage API (LocalStorage). MDN Web Docs. https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API

[4] Russell, S. and Norvig, P. (2020). Artificial Intelligence: A Modern Approach, 4th ed. Pearson.

[5] Minimax Algorithm in Game Theory. GeeksforGeeks. https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-1-introduction/

[6] Alpha-Beta Pruning. GeeksforGeeks. https://www.geeksforgeeks.org/minimax-algorithm-in-game-theory-set-4-alpha-beta-pruning/

[7] SVG — Scalable Vector Graphics. W3C. https://www.w3.org/Graphics/SVG/

[8] Progressive Web Apps (PWA). Google Developers. https://web.dev/progressive-web-apps/

Thank You

Retro Game Hub | Group 3 | BTech CSIT (Cyber Security)

Dristhi Galani · Tanvee Thakre · Atharva Tirkhunde · Rishit Kinge · Shubham Salunke

Under the Guidance of Prof. Rahul Raut

School of Computer Science and Information Technology
Symbiosis Skills and Professional University, Pune | AY 2025-26